

WordFinder: 5-Word Combinatorial Optimization Engine

Technical Overview & Architecture Documentation

WordFinder is a high-performance, multithreaded C++ solver designed to find all unique combinations of 5 words (containing 5 distinct letters each) that perfectly map to 25 unique letters of the English alphabet. The project provides an optimized solution to the popular combinatorial challenge featured across computer science communities.

Technical Implementation & Optimizations

To solve a combinatorial search problem of this magnitude efficiently, the engine avoids naive string allocations, heavy collection copying, and nested character iterations. Instead, the architectural design relies on low-level bitwise manipulation and load-balanced parallel processing to maximize CPU execution throughput.

1. O(1) Bitmask Comparisons

Instead of utilizing a costly, nested character-by-character check which results in $O(n^2)$ time complexity, each word is mapped to a single 32-bit integer identifier serving as its unique letter bitmask:

- **Bit Allocation:** Each bit within the bitmask corresponds directly to a lowercase letter in the alphabet, where *bit 0* = 'a' and *bit 25* = 'z'.
- **Filtering Stage:** The input dictionary (*words_alpha.txt*) is filtered strictly upon loading to include only words containing exactly 5 characters, zero repeating characters, and zero unique anagram duplicates (keeping only the first alphabetical permutation).
- **Intersection Check:** Determining if two words share an overlapping letter simplifies to a single CPU clock instruction via a bitwise intersection check:

$$mask_A \& mask_B == 0$$

2. Pruned Depth-First Search (DFS)

Combinations are discovered by traversing a tree-based Depth-First Search structure, where each separate branch begins with a distinct root word forward-allocated from the vocabulary array.

- Child nodes are conditionally appended to the current traversal path if and only if their precomputed bitmask shares no common set bits with the cumulative bitmask of the entire upstream traversal path.
- **Permutation Deduplication:** Words are strictly evaluated and inserted forward in the list index sequence. This structural constraint eliminates redundant search paths and duplicate permutations entirely. Once a branch reaches a depth of 5, the valid set is pushed instantly to the results collection.

3. Static Workload Load-Balancing

Initially, a single-threaded execution took roughly 15 seconds to evaluate all valid combinations. To maximize hardware utilization across modern multi-core systems, the engine parallelizes the search space across multiple worker threads:

- **The Challenge:** The search space workload distribution is heavily front-loaded. Early root indices generate massively larger search trees than indices toward the tail end of the word list, causing classic thread starvation if divided naively.
- **The Solution:** Instead of splitting the root list evenly by index count, the engine pre-computes the expected number of comparisons across the structural index curve and divides that mathematical workload surface area evenly across available CPU cores.
- **Lock Minimization:** Worker threads operate completely independently within their designated search spaces, only acquiring a mutex lock when flushing their local result streams to the final shared output file buffer.

Performance Result: Implementing strategic load-balancing coupled with bitwise state matching successfully reduced operational execution runtime from **~15 seconds** down to **~3.5 seconds** (yielding a **~4.2×** performance scale-up on standard multicore architectures).



How to Run the Program

The system is compiled into a highly optimized native binary executable. Follow these steps to run the benchmark performance modes:

1. Navigate to the project build output directory:

```
cd WordFinder/x64/Release/
```

2. Execute the compiled binary utility:

```
./WordFinder.exe
```

3. Select your desired execution mode via the console prompt interface:

- Press 1 for Multi-Threaded execution.
- Press 2 for Single-Threaded execution.

Console Output Execution Example

```
Select run mode: 1) Multi Threaded 2) Single Threaded Enter choice: 1 ***** Runtime: 3715 [ms] *****
```

File Assets & Solutions Layout

- **Input Dataset:** Resources/words_alpha.txt
- **Computed Output:** Resources/solution.txt (Contains all 538 valid 5-word combinations discovered)

The complete Microsoft Visual Studio solution structure can be inspected by opening the WordFinder.sln project file inside the root base directory layout.